



भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

ID1063: Introduction to Programming (Lab-5)

Srijith P.K, **Saurabh Kumar**

Dept of CSE

IIT Hyderabad

Lab-5 Questions

Three Questions

Don't forgot to enable
ITP-Monitor Plugin

Q1: Press duration tracker

The year is 2075. At the futuristic campus of IIT Hyderabad, the Introduction to Programming (ITP) students have been recruited for a special mission. The Government of India's Space Security Agency (SSA) is in danger: enemy spies transmit secret codes using experimental terminals. Unlike ordinary systems, these terminals do not merely record the letters typed — they record the precise system clock time (in milliseconds) of each keystroke.

Every spy's code word is special: It is made of distinct characters (no repetition so that no spy can hide patterns). The order and timing of typing reveal behavioral fingerprints — stress, hesitation, and precision. The SSA has partnered with IIT Hyderabad to uncover these patterns, asking its ITP students to analyze keystrokes.

You are now part of this mission. Your job is to write a program that will process the spy's keystroke data.

You will be given the following information:

- ☐ The number of characters typed.
- ☐ The exact timestamps (in ms) when each character was pressed.
- ☐ The secret string of characters (all unique).

Your job is to calculate the duration of each character and print it out.

Now with this help SSA analysts will then use this data to reconstruct the spy's typing rhythm and validate whether the message is authentic. Failing to implement this correctly could mean the spies remain undetected... so write carefully.

Q1: Press duration tracker

Input format: Input will be in three separate lines, and is as follows:

- ❑ The first line contains an integer n , representing the size of the timestamps array and the string size (number of keystrokes).
- ❑ The second line contains n **long long int** separated by a space character, representing the timestamps(ms) of each keystroke.
- ❑ The third line contains a string of characters of length n .

Note: Make sure all the characters in the input string are distinct.

Output format: Print each character and its duration separated by a space and terminated by a newline. Therefore output contains n lines.

Q1: Press duration tracker

Example:

Input

7

2 5 8 9 10 11 12

student

Output

s 2

t 3

u 3

d 1

e 1

n 1

t 1

Explanation

- Initially timer starts from 0 milliseconds
- The character s is typed for a duration of $2 \text{ ms} - 0 \text{ ms} = 2 \text{ ms}$
- The character t is typed for a duration of $5 \text{ ms} - 2 \text{ ms} = 3 \text{ ms}$
- The character u is typed for a duration of $8 \text{ ms} - 5 \text{ ms} = 3 \text{ ms}$
- The character d is typed for a duration of $9 \text{ ms} - 8 \text{ ms} = 1 \text{ ms}$
- The character e is typed for a duration of $10 \text{ ms} - 9 \text{ ms} = 1 \text{ ms}$
- The character n is typed for a duration of $11 \text{ ms} - 10 \text{ ms} = 1 \text{ ms}$
- The character l is typed for a duration of $12 \text{ ms} - 11 \text{ ms} = 1 \text{ ms}$

Input Constraints:

- $2 \leq n \leq 20$
- All the elements in the array are positive and strictly increasing i.e each and every element in the array is strictly greater than its preceding element.

Q2: Basic Array Ops - 1

Recently, in an ITP Theory class at IITH, students were introduced to the concept of arrays and their practical applications. One of the examples discussed was that arrays can be effectively used to store student records, such as marks scored in a course. In this system, the array index represents the student roll number, while the value at that index represents the marks scored by that student. Once the marks are stored, the course instructor explained that various operations can be performed on these records to implement relative grading.

The instructor highlighted that relative grading often involves a process called normalization of marks, where students' scores are analyzed in relation to the group. To get a basic understanding of normalization, students realized there are four essential operations they must practice:

- ❑ **Searching:** Checking whether a given mark exists in the list of marks.
- ❑ **Average:** Finding the average score of the class, rounded to two decimal points.
- ❑ **Minimum:** Identifying the lowest score in the class, rounded to two decimal points.
- ❑ **Maximum:** Identifying the highest score in the class, rounded to two decimal points.

Through this exercise, students not only learned about arrays but also understood how basic mathematical operations can be combined with them to solve real-life academic problems.

Q2: Basic Array Ops - 1

Input Format: Input will be in three lines as follows:

- ❑ The first line contains an integer n , representing the number of students.
- ❑ The second line contains n floating point numbers (float) separated by spaces, denoting the marks of each student.
- ❑ The third line contains an integer k , denoting the operation to be performed.

Output Format: Output will contain only a single line and print it as follows:-

- ❑ If $k = 1$, find whether 100 exists in the array, and print 1 if it exists or else -1.
- ❑ If $k = 2$, find the average marks of the class. Where the average is defined as the total marks of the students divided by the total number of students.
- ❑ If $k = 3$, find the minimum marks scored by the student and print the minimum mark.
- ❑ If $k = 4$, find the maximum marks scored by the student and print the maximum mark.

Constraints:

Q2: Basic Array Ops - 1

Examples:

S.No.	Input	Output	Explanation
1	5 10.5 6.25 14 15 13 1	-1	100 does not exist in the given array.
2	5 10.5 6.25 14 15 100 1	1	100 exist in the given array.
3	5 10.5 6.25 14 15 100 2	29.15	The total of all the given marks is 145.75. Upon division with input size $n=5$ we obtain an average of 29.15. Note round the final answer to two decimal places.
4	5 10.5 6.25 14 15 100 3	6.25	6.25 is the minimum mark attained among all the students. Note round of the final answer to two decimal digits.
5	5 10.5 6.25 14 15 100 4	100.00	100 is the maximum mark attained among all the students. Note round of the final answer to two decimal digits.

Q3: Basic Array Ops - 2

At IITH, Sasha and Saini had just finished their ITP Theory class where arrays were taught with real-world applications. Excited, they decided to explore arrays more deeply in the lab.

Sasha said: “We always use arrays just to store marks. But arrays can actually do so much more. Let’s try new experiments!”

Saini agreed: “Yes! But I feel coding them correctly might be tricky. Maybe we should ask for help in writing the code.”

Together, they came up with four interesting operations they wanted to perform on arrays of marks:

- ❑ **Reversing:** Reverse the order of marks.
- ❑ **Frequency:** How many times each mark is occurring in the given marks array.
- ❑ **Unique Elements:** Identifying all the marks in the given array which had occurred only once in the array.
- ❑ **Duplicate Elements:** Identifying all the marks in the given array which had occurred more than once in the array.

But since Sasha and Saini were finding it difficult to code these operations, they have asked you to help them implement these problems in code. To make things simple they assumed all the marks are integers.

Q3: Basic Array Ops - 2

Input Format: Input will be in three lines as follows:

- ❑ First line contains an integer n , representing the number of students.
- ❑ Second line contains n integers separated by spaces, denoting the marks of each student.
- ❑ Third line contains an integer k , denoting the operation to be performed.

Output Format: Prints the output as follows:

- ❑ If $k = 5$, reverse the order of marks in the given array and print them.
- ❑ If $k = 6$, calculate how many times each mark is occurring in the given array and print them as mark followed by its occurrence separated by space and terminate with a newline.
- ❑ If $k = 7$, identify the unique elements in the array and print them. If there exists no unique marks then print -1.
- ❑ If $k = 8$, identify the duplicate elements in the array and print them. If there exists no duplicates marks then print -1.

Constraints:

Q3: Basic Array Ops - 2

Examples:

S.No.	Input	Output	Explanation
1	5 1 2 3 4 5 5	5 4 3 2 1	Printing in reverse order of the marks from last index to first index. Note - after printing the result terminate the output with a newline character, and separate each number by a space.
2	5 1 2 2 4 5 6	1 1 2 2 4 1 5 1	1, 4, 5 occur only once, while two occurs twice. Note: after printing the result, terminate the output with a newline character
3	5 1 2 3 4 5 6	1 1 2 1 3 1 4 1 5 1	All the elements occur only once. Note: after printing the result, terminate the output with a newline character.
4	5 1 2 3 4 5 7	1 2 3 4 5	All the marks found in the array occur only once. Hence, all the marks are considered to be unique and got printed and terminated with the new line character.

Q3: Basic Array Ops - 2

Examples:

S.No.	Input	Output	Explanation
5	5 1 2 2 4 5 7	1 4 5	The occurrence of 2 is more than once, so it is not unique. The rest of the unique elements are printed and terminated by newline.
6	5 1 2 2 2 1 7	-1	The occurrence of all the marks is more than once, so there is no unique mark. Hence, -1 is printed and terminated by newline character.
7	5 1 2 3 4 5 8	-1	The occurrence of all the marks is unique, which implies there are no duplicates. Hence, -1 is printed and terminated by a newline character.
8	5 1 2 2 4 5 8	2	The occurrence of 2 is not unique, and the rest of them are unique, and hence only 2 got printed and terminated by newline character.

Disclaimer: Just print the marks in the order by which they had appeared in the array. We do not need any modifications like sorting.